

Voyager's Artifacts

Theoretical background

The voyagers found unusual artifacts – square stone tablets divided into grids and filled with numbers. Unfortunately, some of the spaces were badly worn away. However, the explorers noticed that wherever the numbers were still legible, the sum of the rows and columns was always the same. They also saw that certain digits repeated within the rows or columns. Therefore, they decided to find a way to restore the missing spaces.

Goal

Your task is to create a module that restore missing values in the cells of a square table so that the sum of all the elements in the columns and rows is the same. Every cell is one byte. The missing value is represented by the number 255 (0xFF). In an input block of data, you receive a square table row by row. The size of the input data block and the square size are unknown at the beginning, but maximum 4096B (64x64).

As a result, the entire square must be sent back, possibly restored.

Figure 1 Example of a restorable square

	0	1	2	3	4	5	6	7	8	
0										N
1										N
2										N
3										N
4										N
5										N
6										N
7										N
8										N
	N	N	N	N	N	N	N	N	N	SUM

Figure 2 Example of a restorable square

	0	1	2	3	
0					5
1					5
2					5
3					5
	5	5	5	5	SUM

During the field restoration process, it may happen that the square is not uniquely defined and it is unrestorable. In this case you just send back what you got as a result.

Figure 3 Example of an unrestorable square

	0	1	2	3	4	5	
0							N
1							N
2							N
3							N
4							N
5							N
	N	N	N	N	N	N	SUM

Provided module

You can edit only the *task_11.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 1 List of inputs and outputs of the task module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	8 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	8 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet the task receives a sequence of data on the *i_data* port. The maximal amount of data is 4KB. The beginning of the data block is marked by the *i_first* signal and the end by the *i_last* signal. Throughout the data transfer, the *i_valid* signal is high.

Task receives data in the way shown in the figure below:

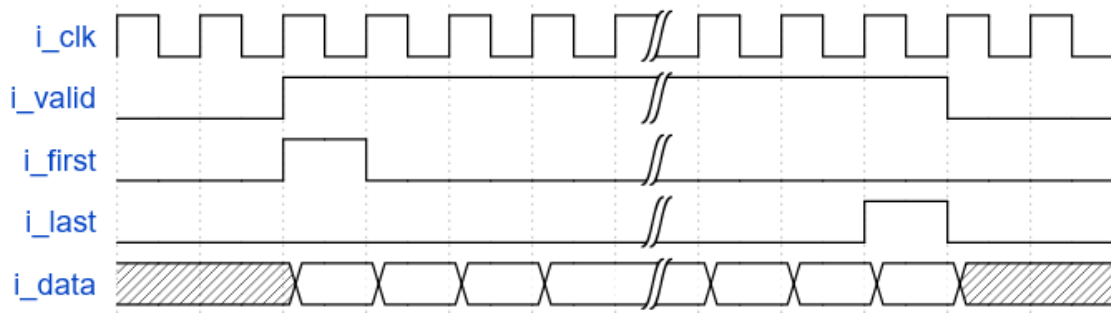


Figure 4 Input waveforms.

To send data from the task in a proper way one needs to provide the *o_valid* signal that is synchronized with data on the *o_data* bus and *o_last* signal that is synchronized with the last sample in output packet.

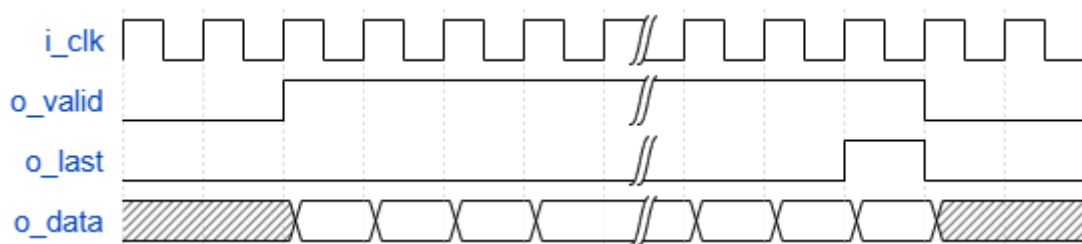


Figure 5 Output waveforms.

Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing *o_valid* and *o_last* signals.

Evaluating the task

The evaluation is done by comparing the sequence returned by the task with the expected sequence calculated by the reference model in the Judge. If the sequences are the same, the task is marked as correctly solved.

In Development mode, the task will be tested using a single set of data that is randomized on each run. In Evaluation mode, the task will be tested using 3 fixed sets of data, which remain the same across all runs and are identical for every team (for more information, check chapter 2.3.1 **Testing modes** in the **FPGA_Hackathon_2026_Project_Guide** document).

You earn 5 points for correctly processing each test vector. In the Development mode, this value simply indicates that the task passed the test. In the Evaluation mode, your total base score is calculated by multiplying the number of points by the number of test vectors (3), so you can earn up to 15 points.

Additional points (up to 5) may be awarded for **resource utilization when latency is below 85µs (8122 cycles)** (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).